

Correction du TP 1 d'Interprétation et compilation

Christian Rinderknecht

23 et 24 septembre 2003

1 Questions

1. Implantez la calculette présentée dans le cours sans tenir compte des cas d'erreurs, puis traitez ceux-ci à l'aide des exceptions O'Caml.
2. Ajoutez maintenant la liaison locale et les variables. Identifiez les erreurs possibles et ajoutez leur gestion. Testez des cas d'évaluation correcte et des cas provoquant ces erreurs. (*Gardez une trace de vos cas de test.*) Par exemple :

```
1+2*(ifz 0 then let x = 0 in x+3 else 4)
```

3. On souhaite se doter d'une conditionnelle *sans ajout des booléens*, dans le style du langage C :

- **Syntaxe concrète**

```
Expression ::= ...  
| "ifz" Expression "then" Expression "else" Expression
```

- **Exemple** 1+2*(ifz 0 then 3 else 4)

- **Syntaxe abstraite**

```
type expr = ... | lfz of expr × expr × expr;;
```

Donnez une sémantique opérationnelle pour cette conditionnelle et complétez votre évaluateur par son implantation.

4. Enrichissez la calculette par les booléens, les opérateurs logiques de conjonction et disjonction, ainsi que d'une conditionnelle dont la condition est booléenne. Un lexique et une grammaire sont respectivement proposés dans les fichiers `lexer.mll` et `parser.mly`. Proposez une sémantique opérationnelle et une implantation en O'Caml.
5. Le codage fonctionnel pour les environnements est très inefficace car, dans le pire des cas, pour obtenir la valeur d'une variable il faut évaluer autant d'appels de fonction qu'il y a de variables dans l'environnement, et un appel de fonction est très coûteux, en général. Il est plus judicieux, dans un premier temps, de remplacer les appels de fonction par des accès dans une liste. Ainsi un environnement est une liste qui associe les variables aux valeurs. La recherche d'une valeur se

fait alors à l'aide de la fonction `List.assoc` : $\forall \alpha, \beta. \alpha \rightarrow (\alpha \times \beta) \text{ list} \rightarrow \beta$.
Conservez un exemplaire de `eval.ml` et réécrivez-le en implantant ce schéma et testez à nouveau. (*Gardez une trace de vos cas de test.*)

2 Réponses

2.1 Module Eval

Nous donnons ici le contenu du module `Eval` (fichier `eval.ml`). Il contient plusieurs sous-modules correspondant chacun à une calculette avec différentes constructions qui sont ajoutées au fur et à mesure des questions.

```
open Ast

module Initial = struct
  type value = int

  exception DivByZero

  let rec eval e = match e with
    Const n -> n
  | BinOp (op,e1,e2) ->
      let v1 = eval e1 and v2 = eval e2
      in begin match op with
          Add -> v1 + v2 | Sub -> v1 - v2 | Mult -> v1 * v2
        | Div -> if v2 = 0 then raise DivByZero else v1/v2
        end
    | _ -> failwith "Construction non traitée"
  end

  module WithLet = struct
    type value = int

    type environnement = string -> value

    let empty_env = fun x -> raise Not_found

    let extend env (x,v) = fun y -> if x = y then v else env y

    type error =
      DivByZero
    | FreeVar of string

    exception Err of error

    let rec eval env e = match e with
      Const n -> n
    | BinOp (op,e1,e2) ->
```

```

    let v1 = eval env e1 and v2 = eval env e2
    in begin match op with
        Add -> v1 + v2 | Sub -> v1 - v2 | Mult -> v1 * v2
        | Div -> if v2 = 0 then raise (Err DivByZero) else v1/v2
    end
| Var x -> begin try env x with
    Not_found -> raise (Err (FreeVar x))
end
| Let (x,e1,e2) ->
    let v1 = eval env e1 in eval (extend env (x,v1)) e2
| _ -> failwith "Construction non traitée"
end

```

```

module WithCondInt = struct
  type value = int
  type environnement = string -> value

  let empty_env = fun x -> raise Not_found
  let extend env (x,v) = fun y -> if x = y then v else env (y)

  type error =
    DivByZero
  | FreeVar of string

  exception Err of error

```

```

let rec eval env e = match e with
  Const n -> n
| BinOp (op,e1,e2) ->
    let v1 = eval env e1 and v2 = eval env e2
    in begin match op with
        Add -> v1 + v2 | Sub -> v1 - v2 | Mult -> v1 * v2
        | Div -> if v2 = 0 then raise (Err DivByZero) else v1/v2
    end
| Var x -> begin try env x with
    Not_found -> raise (Err (FreeVar x))
end
| Let (x,e1,e2) ->
    let v1 = eval env e1 in eval (extend env (x,v1)) e2
| Ifz (e1,e2,e3) ->
    if eval env e1 = 0 then eval env e3 else eval env e2
| _ -> failwith "Construction non traitée"
end

```

```

module WithCond = struct
  type value = int
  type environnement = string -> value

```

```

let empty_env = fun x -> raise Not_found
let extend env (x,v) = fun y -> if x = y then v else env y

type error =
  DivByZero
| FreeVar of string

exception Err of error

let rec eval_bool b = match b with
  True -> true
| False -> false
| And (b1,b2) -> (eval_bool b1) && (eval_bool b2)
| Or (b1,b2) -> (eval_bool b1) || (eval_bool b2)
| Not b -> not (eval_bool b)

let rec eval env e = match e with
  Const n -> n
| BinOp (op,e1,e2) ->
  let v1 = eval env e1 and v2 = eval env e2
  in begin match op with
    Add -> v1 + v2 | Sub -> v1 - v2 | Mult -> v1 * v2
    | Div -> if v2 = 0 then raise (Err DivByZero) else v1/v2
  end
| Var x -> begin try env x with
  Not_found -> raise (Err (FreeVar x))
end
| Let (x,e1,e2) ->
  let v1 = eval env e1 in eval (extend env (x,v1)) e2
| Ifz (e1,e2,e3) ->
  if eval env e1 = 0 then eval env e3 else eval env e2
| If (b,e1,e2) ->
  if eval_bool b then eval env e1 else eval env e2
| _ -> failwith "Construction non traitée"
end

module WithEnvList = struct
  type value = int

  type environnement = (string * value) list

  type error =
    DivByZero
  | FreeVar of string

  exception Err of error

  let lookup x env =
    try List.assoc x env with

```

```

    Not_found -> raise (Err (FreeVar x))

let rec eval_bool b = match b with
  True -> true
| False -> false
| And (b1,b2) -> (eval_bool b1) && (eval_bool b2)
| Or (b1,b2) -> (eval_bool b1) || (eval_bool b2)
| Not b -> not (eval_bool b)

let rec eval env e = match e with
  Const n -> n
| BinOp (op,e1,e2) ->
  let v1 = eval env e1 and v2 = eval env e2
  in begin match op with
    Add -> v1 + v2 | Sub -> v1 - v2 | Mult -> v1 * v2
  | Div -> if v2 = 0 then raise (Err DivByZero) else v1/v2
  end
| Ifz (e1,e2,e3) ->
  if eval env e1 = 0 then eval env e3 else eval env e2
| If (b,e1,e2) ->
  if eval_bool b then eval env e1 else eval env e2
| Var x -> lookup x env
| Let (s,e1,e2) -> let v1 = eval env e1 in eval ((s,v1)::env) e2
| _ -> failwith "Construction non traitée"
end

```

2.1.1 Sous-module Eval.Initial

Le sous-module Eval.Initial regroupe les définitions de type et de valeurs correspondant à la calculatrice avec constantes et opérateurs arithmétiques. Il répond à la question 1.

L'exception DivByZero sera déclenchée lorsqu'une division par zéro sera anticipée par la calculatrice.

Le dernier cas du filtre de la fonction eval est nécessaire car la syntaxe abstraite, c'est-à-dire le type Ast.expr, inclut tous les constructeurs pour réaliser l'ensemble du travail pratique.

2.1.2 Sous-module Eval.WithLet

Le module Eval.WithLet reprend le module Eval.Initial et l'étend avec les variables et la liaison locale. Il répond à la question 2.

Les environnements ρ sont codés de façon fonctionnelle.

La fonction empty_env implante l'environnement vide.

La fonction extend implante l'extension des environnements par une nouvelle liaison (il s'agit de l'opérateur $\oplus$). Ainsi l'implantation de $\rho \oplus x \mapsto v$ est extend env (x,v).

Il y a maintenant deux types d'erreurs : la division par zéro et les variables libres (c'est-à-dire non introduites par un `let`). Il est alors utile de regrouper ces erreurs sous un même type. Ainsi, l'exception `Err (FreeVar x)` est déclenchée quand la variable `x` est libre dans l'expression. Par exemple : `let a = 1 in b` (ici la variable libre est `b`).

La fonction `eval` prend ici un argument supplémentaire : l'environnement dans lequel l'expression doit être évaluée.

2.1.3 Sous-module `Eval.WithCondInt`

Le module `Eval.WithCondInt` reprend et étend le module `Eval.WithLet` avec une construction conditionnelle discriminant les branches avec un entier (à zéro ou non), dans le style du langage C. Il répond à la question 3. On nous donne :

- **Syntaxe concrète**
`Expression ::= ...`
`| "ifz" Expression "then" Expression "else" Expression`
- **Exemple** `1+2*(ifz 0 then 3 else 4)`
- **Syntaxe abstraite**
`type expr = ... | ifz of expr * expr * expr`

Une sémantique opérationnelle pour notre conditionnelle est :

$$\frac{\rho \vdash e_1 \rightarrow \text{Int}(n) \quad n \neq 0 \quad \rho \vdash e_2 \rightarrow v_2}{\rho \vdash \text{ifz}(e_1, e_2, e_3) \rightarrow v_2} \text{ IF-THEN}$$

$$\frac{\rho \vdash e_1 \rightarrow \text{Int}(0) \quad \rho \vdash e_3 \rightarrow v_3}{\rho \vdash \text{ifz}(e_1, e_2, e_3) \rightarrow v_3} \text{ IF-ELSE}$$

L'implantation par un nouveau cas dans le filtre de la fonction `eval` est immédiat.

2.1.4 Sous-module `Eval.WithCond`

Le module `Eval.WithCond` reprend le module `Eval.WithCondInt` et l'étend avec une conditionnelle qui discrimine ses branches à l'aide d'une expression booléenne (comme en O'Caml, par exemple). Il répond à la question 4. Nous devons ajouter au langage de la calculette les expressions booléennes. Pour faciliter le travail, cela est déjà fait au niveau de la syntaxe concrète et abstraite – cf. `lexer.mll`, `parser.mly` et `ast.ml`. Dans ce dernier, la syntaxe abstraite contient la définition des booléens.

Voici les syntaxes reconnues par défaut par la calculette :

- **Syntaxe concrète**
`Boolean ::= true | false`
`| Boolean "and" Boolean`
`| Boolean "or" Boolean`

```

      | "not" "(" Boolean ")"
Expression ::= ...
      | "if" Boolean "then" Expression "else" Expression

```

- **Exemple** `1+2*(if true then 3 else 4)`

- **Syntaxe abstraite**

```

type boolean = True | False | Not of boolean
      | And of boolean × boolean
      | Or of boolean × boolean

```

```

type expr = ... | lf of boolean × expr × expr

```

On remarque que la solution présentée ici ne permet pas qu'un booléen soit une expression, donc le type des valeurs restera inchangé dans la sémantique, mais il faudra une seconde relation d'évaluation dédiée aux booléens. Donc, en théorie, il faudrait choisir une autre notation que $\rho \vdash e \rightarrow v$, mais en pratique c'est commode de reprendre la même, à ceci près que les expressions booléennes n'ont pas besoin d'un environnement pour s'évaluer car nous n'avons pas permis au `let ... in ...` de lier des booléens. Donc nous noterons la relation d'évaluation des booléens $b \rightarrow \bar{b}$, où b est une valeur OCaml de type `Ast.bexpr` et $\bar{b}$ est de type `bool`.

$$\frac{b \rightarrow \mathbf{true} \quad \rho \vdash e_2 \rightarrow v_2}{\rho \vdash \mathbf{lf}(b, e_2, e_3) \rightarrow v_2} \text{ IF-THEN} \qquad \frac{b \rightarrow \mathbf{false} \quad \rho \vdash e_3 \rightarrow v_3}{\rho \vdash \mathbf{lf}(b, e_2, e_3) \rightarrow v_3} \text{ IF-ELSE}$$

L'implantation de cette sémantique est directe (elle s'appuie inductivement sur la syntaxe abstraite). l'implantation par un nouveau cas dans le filtre de la fonction `eval` est immédiat.

2.1.5 Sous-module `Eval.WithEnvList`

Le module `Eval.WithEnvList` reprend le module `Eval.WithCond` en remplaçant le codage fonctionnel de l'environnement par un codage à l'aide d'une structure de donnée nommée *liste d'association*. Il répond à la question 5. Une liste d'association est une liste de paires qui modélisent les liaisons. Ainsi, $\rho(x)$ est implanté non plus par `env ×` (`env` est une fonction OCaml) mais par `List.assoc × env` (`env` est une liste d'association). La raison est essentiellement l'efficacité de la recherche d'une liaison, bien qu'il serait encore plus efficace d'utiliser une autre structure de donnée, comme une table de hachage, si le nombre de variable est élevé en moyenne par programme.

Les environnements ρ sont codés à l'aide d'une liste d'association. L'environnement vide est alors simplement codé par `[ ]` (plus besoin de `empty_env`).

Au lieu de coder $\rho(x)$ par `env ×` (plus un **try with**, on fait `lookup × env`).

L'ajout d'une nouvelle liaison devient l'ajout d'une paire au tête de liste : $\rho \oplus x \mapsto v$ est codé par `(x,v) : env` au lieu de `extend env (x,v)` avec un codage fonctionnel. On n'a donc plus besoin de la fonction `extend`.

Dans la définition de la fonction `eval`, le motif `Var x` change et appelle `lookup x env` dorénavant, et le cas de la liaison locale est plus simple.

2.2 Module Main

Le module `Main` est le pilote de la calculatrice. C'est ce module *uniquement* qui interagit avec l'utilisateur et, en particulier, filtre les erreurs des autres modules (ici `Eval`). Il est composé de plusieurs sous-modules correspondant chacun à un des sous-modules de `Eval` de même nom. Par exemple `Main.Initial` correspond à (en l'occurrence *utilise*) `Eval.Initial`.

```
module Initial = struct
```

```
  open Eval.Initial
```

```
  let calc ast =
```

```
    try
```

```
      print_int (eval ast); print_newline(); exit 0
```

```
    with DivByZero ->
```

```
      print_endline "Error: division by zero."; exit 1
```

```
end
```

```
module WithLet = struct
```

```
  open Eval.WithLet
```

```
  let calc ast =
```

```
    try
```

```
      print_int (eval empty_env ast); print_newline(); exit 0
```

```
    with Err DivByZero ->
```

```
      print_endline "Error: division by zero."; exit 1
```

```
    | Err (FreeVar x) ->
```

```
      print_endline ("Error: free variable " ^ x); exit 1
```

```
end
```

```
module WithCondInt = struct
```

```
  open Eval.WithCondInt
```

```
  let calc ast =
```

```
    try
```

```
      print_int (eval empty_env ast); print_newline(); exit 0
```

```
    with Err DivByZero ->
```

```
      print_endline "Error: division by zero."; exit 1
```

```
    | Err (FreeVar x) ->
```

```
      print_endline ("Error: free variable " ^ x); exit 1
```

```
end
```

```
module WithCond = struct
```



```

open Eval.WithCond

let calc ast =
  try
    print_int (eval empty_env ast); print_newline(); exit 0
  with Err DivByZero ->
    print_endline "Error: division by zero."; exit 1
  | Err (FreeVar x) ->
    print_endline ("Error: free variable " ^ x); exit 1
end

module WithEnvList = struct

  open Eval.WithEnvList

  let calc ast =
    try
      print_int (eval [] ast); print_newline(); exit 0
    with Err DivByZero ->
      print_endline "Error: division by zero."; exit 1
    | Err (FreeVar x) ->
      print_endline ("Error: free variable " ^ x); exit 1
  end

  let main () =
    let () =
      print_endline "Saisissez une expression + RET + CTRL-D:";
      flush stdout in
    let lexbuf = Lexing.from_channel stdin in
    let ast = Parser.expression Lexer.token lexbuf in
    let () = print_string "=>"
  in WithEnvList.calc (ast)

  let _ = Printexc.print main ()
end

```

2.2.1 Sous-module Main.Initial

Le sous-module Main.Initial correspond au cas initial avec les quatre opérations arithmétiques de base ainsi que les constantes. Il répond donc à la question 1 en complétant le module Eval.Initial (cf. section 2.1.1). La seule erreur qui puisse provenir de Eval.Initial est DivByZero.

2.2.2 Sous-module Main.WithLet

Le sous-module Main.WithLet correspond au module Eval.WithLet (cf. section 2.1.2). En complétant ce dernier, il répond ainsi à la question 2. Les

variables et la liaison locale sont possibles, donc il faut filtrer en plus le cas des variables libres.

2.2.3 Sous-module **Main.WithCondInt**

Le sous-module `Main.WithCondInt` correspond à `Eval.WithCondInt` (cf. section 2.1.3). En complétant ce dernier, il répond ainsi à la question 3. Du point de vue du pilotage, rien ne change : les erreurs possibles sont toujours les mêmes qu’avec `Main.WithLet`.

2.2.4 Sous-module **Main.WithCond**

Le sous-module `Main.WithCond` correspond à `Eval.WithCond` (cf. section 2.1.4). En complétant ce dernier, il répond ainsi à la question 4.

2.2.5 Sous-module **Main.WithEnvList**

Le sous-module `Main.WithEnvList` correspond à `Eval.WithEnvList` (cf. section 2.1.5). En complétant ce dernier, il répond ainsi à la question 5. Nous devons néanmoins réécrire `Main.WithEnvList.calc` car le codage de l’environnement vide a changé (c’est `[]` maintenant).

La fonction `main` est celle qui est appelée après le lancement du programme. Pour tester en boîte blanche (c.-à-d. en modifiant le code) il faut alors modifier les appels à `calc` et recompiler. Par exemple, nous avons ici l’appel à `WithEnvList.calc`, que l’on peut remplacer par un appel à `WithCond.calc` etc. Cette méthode n’est pas bonne car, en général, un test en boîte grise ou noire est préférable. Pour cela il faudrait proposer, par exemple, à l’usager des options sur la ligne de commande de lancement de la calculatrice.

Cette liaison globale est le point d’entrée du programme (il s’agit d’évaluer l’appel à `main`). Son intérêt est de demander à l’environnement d’exécution de tracer toutes les exceptions qui pourraient remonter de l’appel à `main`.